Each item in the list is passed into the provided function, and the return value of each function is returned inside of a new list. The number of elements in the list does not change: if five elements are in the input list, five elements are returned.

Enum.reduce

Finally, Enum.reduce/3 iterates over each value in a list and transforms the accumulator. The accumulator can be any value that you want. Here's an example of a reduce function that finds the largest value in a list:

```
$ iex
iex> Enum.reduce([1, 2, 3, 1], 0, fn num, accumulator ->
  if num > accumulator do
    num
  else
    accumulator
  end
end)
```

```
3
```

The return value of each iteration becomes the next iteration's accumulator. You can build up any result that you want with this technique. You could build a new map, modify a custom struct, or keep track of a single value or multiple values.

Here's the same code in Ruby:

```
$ irb
irb(main)> [1, 2, 3, 1].reduce(0) do |accumulator, num|
  num > accumulator ? num : accumulator
end
```

Functionally, these are similar. But the actual syntax between Ruby and Elixir is quite different.

reduce takes a bit of time to get used to. But once you do, it becomes an important tool for iterating over data structures.

Comprehensions

Comprehensions[6] are a built-in syntax for iterating and reducing data. Comprehensions use the for keyword, but it's nothing like a traditional for-loop. Comprehensions can iterate a single source of data, combine multiple sources of data, filter data, and reduce data into any result format.

6. https://hexdocs.pm/elixir/Kernel.SpecialForms.html#for/1